

DSA STUDY BLUEPRINT SERIES

126. Bridge in Graph using Tarjans Algorithm Critical Connections

Identifying Connectivity Bridges using Tarjan's DFS

Section 10 • C++ Core Data Structures & Algorithms

Core Concepts & Visual Blueprint

Theoretical models and memory architectures

Key Principles

- **Bridge:** An edge whose removal increases the number of connected components.
- Track discovery times (`disc`) and lowest reachable node (`low`).

Memory & Execution Blueprint

Edge 2-3 is a Bridge because $\text{low}[3] (3) > \text{disc}[2] (2)$

C++ Implementation Reference

Standard code layout and syntax

```
void dfsBridge(int u, int parent, int& time, vector<int>& adj, vector<int>& disc, vector<int>& low, vector<int>& bridges) {
    disc[u] = low[u] = ++time;
    for (int v : adj[u]) {
        if (v == parent) continue;
        if (disc[v] == -1) {
            dfsBridge(v, u, time, adj, disc, low, bridges);
            low[u] = min(low[u], low[v]);
            if (low[v] > disc[u]) bridges.push_back({u, v}); // Bridge!
        } else low[u] = min(low[u], disc[v]);
    }
}
```

Algorithmic Complexity & Best Practices

Performance evaluations and critical guidelines

Performance Table

Aspect / Case	Complexity
Time Complexity	$O(V + E)$
Space Complexity	$O(V)$

Key Practices & Gotchas

- **Important:** Bridge exists if child v cannot reach parent u 's ancestor: `low[v] > disc[u]`.
- Verify boundary conditions (e.g. empty inputs, single elements).
- Optimize dynamic memory allocations to prevent leaks.

Dry Run & Step-by-Step Execution

Trace analysis on a sample input case

Execution Walkthrough

Let's trace Dijkstra distance relaxation updates on active vertices:

Vertex popped	Adjacent edge checked	Current distance value	New path calculation	Relaxation action
U (Dist = 2)	U → V (Weight = 3)	V (Dist = 7)	New Dist = 2 + 3 = 5	Relax: Update V (Dist = 5)

Interview Variations & Optimization Pathways

Common follow-up problems and optimization strategies

Key Variations & Follow-up Questions

- **Shortest Path choices:** BFS finds unweighted short paths; Dijkstra handles positive weights; Bellman-Ford resolves negative edges.
- **Spanning Trees:** Prim's builds MST from a start node; Kruskal's sorts all edges using disjoint sets.
- **Related LeetCode Problems:** #200 Number of Islands, #743 Network Delay Time, #787 Cheapest Flights Within K Stops.